

Σουβενίρ

Ο Αμάρου είναι σε ένα μαγαζί και αγοράζει σουβενίρ. Υπάρχουν N **τύποι** σουβενίρ. Το κατάστημα διαθέτει άπειρα σουβενίρ από κάθε τύπο.

Κάθε τύπος σουβενίρ έχει μια σταθερή τιμή. Συγκεκριμένα, ένα σουβενίρ τύπου i (όπου $0 \leq i < N$) κοστίζει $P[i]$ νομίσματα, όπου $P[i]$ είναι ένας θετικός ακέραιος αριθμός.

Ο Αμάρου γνωρίζει ότι οι τύποι των σουβενίρ είναι ταξινομημένοι σε φθίνουσα σειρά με βάση την τιμή τους και ότι οι τιμές των σουβενίρ είναι διαφορετικές μεταξύ τους. Συγκεκριμένα, $P[0] > P[1] > \dots > P[N-1] > 0$. Επιπλέον, μπόρεσε να μάθει την τιμή του $P[0]$. Δυστυχώς, ο Αμάρου δεν έχει καμία άλλη πληροφορία σχετικά με τις τιμές των σουβενίρ.

Για να αγοράσει κάποια σουβενίρ, ο Αμάρου θα πραγματοποιήσει μια σειρά από συναλλαγές με τον πωλητή.

Κάθε συναλλαγή αποτελείται από τα ακόλουθα βήματα:

1. Ο Αμάρου δίνει στον πωλητή κάποιον (θετικό) αριθμό νομισμάτων.
2. Ο πωλητής βάζει αυτά τα νομίσματα σε μια στοίβα στο τραπέζι στο πίσω δωμάτιο, όπου ο Αμάρου δεν μπορεί να τα δει.
3. Ο πωλητής εξετάζει κάθε τύπο σουβενίρ κατά σειρά από το $0, 1, \dots, N-1$, έναν προς έναν. Κάθε τύπος εξετάζεται **ακριβώς μία φορά** για κάθε συναλλαγή.
 - Όταν εξετάζεται ο τύπος σουβενίρ i , αν το πλήθος των νομισμάτων στη στοίβα είναι τουλάχιστον $P[i]$, τότε
 - ο πωλητής αφαιρεί $P[i]$ νομίσματα από τη στοίβα, και
 - ο πωλητής βάζει ένα σουβενίρ τύπου i στο τραπέζι.
4. Ο πωλητής δίνει στον Αμάρου όλα τα νομίσματα που έχουν απομείνει στη στοίβα και όλα τα σουβενίρ που βρίσκονται στο τραπέζι.

Να σημειωθεί ότι πριν ξεκινήσει μια συναλλαγή δεν υπάρχουν νομίσματα ή σουβενίρ στο τραπέζι.

Βοηθήστε τον Αμάρου να εκτελέσει έναν αριθμό συναλλαγών, έτσι ώστε:

- σε κάθε συναλλαγή να αγοράζει **τουλάχιστον ένα** σουβενίρ, και
- συνολικά να αγοράζει **ακριβώς i** σουβενίρ τύπου i , για κάθε i τέτοιο ώστε $0 \leq i < N$. Προσέξτε ότι αυτό σημαίνει πως ο Αμάρου δεν πρέπει να αγοράσει κανένα σουβενίρ τύπου 0.

Ο Αμάρου δεν χρειάζεται να ελαχιστοποιήσει τον αριθμό των συναλλαγών και έχει απεριόριστο απόθεμα από νομίσματα.

Λεπτομέρειες Υλοποίησης

Θα πρέπει να υλοποιήσετε την ακόλουθη συνάρτηση.

```
void buy_souvenirs(int N, long long P0)
```

- N : το πλήθος των τύπων σουβενίρ.
- $P0$: η τιμή του $P[0]$.
- Αυτή η συνάρτηση καλείται ακριβώς μία φορά για κάθε περίπτωση ελέγχου.

Η παραπάνω συνάρτηση μπορεί να καλεί την ακόλουθη συνάρτηση για να καθοδηγήσει τον Αμάρου να εκτελέσει μια συναλλαγή:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : ο αριθμός των νομισμάτων που δίνει ο Αμάρου στον πωλητή.
- Η συνάρτηση επιστρέφει ένα ζεύγος. Το πρώτο στοιχείο του ζεύγους είναι ένας πίνακας L , που περιέχει τους τύπους σουβενίρ που αγοράζονται (σε αύξουσα σειρά). Το δεύτερο στοιχείο είναι ένας ακέραιος αριθμός R , ο αριθμός των νομισμάτων που επιστρέφονται στον Αμάρου μετά τη συναλλαγή.
- Πρέπει να ισχύει $P[0] > M \geq P[N - 1]$. Η συνθήκη $P[0] > M$ εξασφαλίζει ότι ο Αμάρου δεν αγοράζει κανένα σουβενίρ τύπου 0, και η συνθήκη $M \geq P[N - 1]$ εξασφαλίζει ότι ο Αμάρου αγοράζει τουλάχιστον ένα σουβενίρ. Εάν δεν πληρούνται αυτές οι συνθήκες, η λύση σας θα αποτύχει με το μήνυμα `Output isn't correct: Invalid argument`. Σημειώστε ότι σε αντίθεση με την $P[0]$, η τιμή του $P[N - 1]$ δεν δίνεται στην είσοδο.
- Η συνάρτηση μπορεί να κληθεί το πολύ 5000 φορές σε κάθε περίπτωση ελέγχου.

Η συμπεριφορά του βαθμολογητή **δεν είναι προσαρμοστική** (adaptive). Αυτό σημαίνει ότι η ακολουθία τιμών P καθορίζεται πριν κληθεί η συνάρτηση `buy_souvenirs` και παραμένει σταθερή.

Περιορισμοί

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
- $P[i] > P[i + 1]$ για κάθε i τέτοιο ώστε $0 \leq i < N - 1$.

Υποπροβλήματα

Υποπρόβλημα	Πόντοι	Επιπλέον περιορισμοί
1	4	$N = 2$
2	3	$P[i] = N - i$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ για κάθε i τέτοιο ώστε $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ για κάθε i τέτοιο ώστε $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ για κάθε i τέτοιο ώστε $0 \leq i < N - 1$.
6	33	Χωρίς επιπλέον περιορισμούς.

Παράδειγμα

Έστω η ακόλουθη κλήση.

```
buy_souvenirs(3, 4)
```

Υπάρχουν $N = 3$ τύποι σουβενίρ και $P[0] = 4$. Προσέξτε ότι υπάρχουν μόνο τρεις πιθανές ακολουθίες τιμών P : $[4, 3, 2]$, $[4, 3, 1]$, και $[4, 2, 1]$.

Ας υποθέσουμε ότι η `buy_souvenirs` καλεί την `transaction(2)`. Ας υποθέσουμε ότι η κλήση επιστρέφει $([2], 1)$, που σημαίνει ότι ο Αμάρου αγόρασε ένα σουβενίρ τύπου 2 και ο πωλητής τού επέστρεψε 1 νόμισμα. Προσέξτε ότι αυτό μας επιτρέπει να συμπεράνουμε ότι $P = [4, 3, 1]$, αφού:

- Για $P = [4, 3, 2]$, η `transaction(2)` θα είχε επιστρέψει $([2], 0)$.
- Για $P = [4, 2, 1]$, η `transaction(2)` θα είχε επιστρέψει $([1], 0)$.

Στη συνέχεια, η `buy_souvenirs` μπορεί να καλέσει την `transaction(3)`, η οποία επιστρέφει $([1], 0)$, που σημαίνει ότι ο Αμάρου αγόρασε ένα σουβενίρ τύπου 1 και ο πωλητής τού επέστρεψε 0 κέρματα. Μέχρι στιγμής, συνολικά, έχει αγοράσει ένα σουβενίρ τύπου 1 και ένα σουβενίρ τύπου 2.

Τέλος, η `buy_souvenirs` μπορεί να καλέσει την `transaction(1)`, η οποία επιστρέφει $([2], 0)$, που σημαίνει ότι ο Αμάρου αγόρασε ένα σουβενίρ τύπου 2. Προσέξτε ότι εδώ θα μπορούσαμε επίσης να καλέσουμε την `transaction(2)`. Σε αυτό το σημείο, συνολικά ο Αμάρου έχει αγοράσει ένα σουβενίρ τύπου 1 και δύο σουβενίρ τύπου 2, όπως απαιτείται.

Υποδειγματικός Βαθμολογητής

Μορφή εισόδου:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Μορφή εξόδου:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Εδώ $Q[i]$ είναι το συνολικό πλήθος των σουβενίρ τύπου i που αγοράστηκαν, για κάθε i τέτοιο ώστε $0 \leq i < N$.